

AI Intelligence Ingestion Pipeline

Production-Grade Scalable Architecture — Phase VI

1. System Objective

Build a fault-tolerant ingestion platform that collects startups, products, research papers, AI news and jobs; extracts canonical JSON with LLMs; resolves entities deterministically; and stores traceable records with source URLs and collection timestamps. The design targets 500k+ records without code changes by separating discovery, crawling, extraction, validation and persistence into independently scalable workers.

2. High-Level Architecture

Sources	Discovery / Crawl	Queue	LLM / Rules	Storage
Websites RSS APIs arXiv GitHub	Async HTTP Playwright fallback Rate limiting Dedup	Kafka / SQS partition by source/entity retry queues	Freshness Chunking Gemini → Groq → DeepSeek Entity resolution	PostgreSQL Object storage Vector DB Graph DB

3. Scaling to 500k+ Records

Use horizontally scalable stateless workers. A discovery worker emits normalized URLs/tasks to a durable queue. Crawl workers consume partitions concurrently using asyncio/aiohttp for normal pages and Playwright Async only for JS-rendered or protected high-value sources. LLM workers process extracted text independently, allowing crawl throughput and LLM throughput to scale separately.

Partitioning: partition queue work by source/domain and hash(entity URL). Each task carries source URL, record type, attempt number and idempotency key. **Backpressure:** cap per-domain concurrency and queue depth; slow producers when downstream LLM/storage queues exceed thresholds.

Horizontal scaling: add crawler/LLM worker replicas without changing application logic. Stateless workers store no required progress locally; checkpoints and task state live in durable storage. Object storage holds raw HTML/text so extraction can be replayed without recrawling.

4. Reliability and Fault Tolerance

Every task has bounded retries, exponential backoff with jitter, structured logs, timeout limits and a dead-letter queue. Failures are isolated per URL/record so one bad page cannot terminate a batch. Processing is idempotent using a deterministic record key such as **hash(canonical_url + record_type)**.

Example retry policy: attempt 1 after ~1s, then 2s, 4s, 8s with random jitter, capped at a configured maximum. Permanent 4xx errors are not blindly retried; authentication failures and invalid requests are recorded for review.

AI Intelligence Ingestion Pipeline — Resilience & Data Flow

5. Exact 413 / 429 Handling

HTTP 413 — Payload Too Large: never resend the same oversized prompt unchanged. Estimate token/character size before the request, split dense source text into semantic chunks, and retry with smaller chunks. Preserve document identity and chunk order. Each chunk produces partial structured output; a deterministic merge step combines non-conflicting fields. If a field is absent, keep it null rather than inventing it.

HTTP 429 — Rate Limited: respect Retry-After when supplied; otherwise use exponential backoff plus random jitter. Track provider-specific limits and concurrency. After repeated 429s, temporarily disable that provider for the current run and route new work to the next provider. This prevents a hot provider from blocking the entire pipeline.

Failure	Action	Fallback
413	Reduce chunk size; preserve chunk order; retry.	Smaller chunks / next provider
429	Retry with Retry-After or exponential backoff + jitter.	Provider cooldown → next provider
401/403	Treat as configuration/access failure; do not hot-loop.	Next configured provider
5xx/timeout	Bounded retry with backoff.	Next provider / DLQ

6. LLM Orchestration

Canonical extraction uses a provider chain. The implemented order is Gemini Flash → Groq → DeepSeek when credentials are available. The extractor enforces a fixed JSON schema and a no-hallucination rule: unknown values remain null. Provider health is tracked per run so repeated rate-limit failures do not waste capacity.

Intelligent chunking: clean HTML first, remove scripts/styles/navigation noise, then split content around paragraph/sentence boundaries. Use overlapping boundaries only where needed to avoid losing context. Dense sections are prioritized; chunk metadata records source URL and index.

7. Distributed Freshness and Deduplication

Freshness is evaluated close to ingestion time. Each source has a date normalization adapter that handles ISO timestamps, RSS dates, relative values such as '2 hours ago', and missing metadata. The current news/jobs contract accepts only records whose normalized timestamp is within 24 hours; future or invalid timestamps are rejected. When strict metadata is unavailable, a documented heuristic can be used and marked with lower confidence.

Distributed dedup: generate fingerprints from normalized URL plus record type; for near-duplicate text, use normalized title/entity plus a content hash. Enforce uniqueness in the primary database with an idempotency key and upsert semantics. This makes retries safe across multiple workers.

8. Anti-Bot Strategy

Start with respectful HTTP crawling, source-specific rate limits, caching and retries. Detect Cloudflare/JS-rendered responses and route only high-value pages to Playwright Async. Do not attempt to bypass CAPTCHAs, authentication or access controls. Blocked sources are logged with their status and can be retried later or replaced by an accessible official feed/API.

AI Intelligence Ingestion Pipeline — Storage, Operations & Implementation

9. Storage Architecture

Primary database: PostgreSQL. Store canonical records, source metadata, normalized timestamps, entity IDs, crawl status, retry state and audit information. PostgreSQL provides transactions, unique constraints and upserts needed for deterministic deduplication and idempotent processing.

Raw/object storage: store raw HTML, extracted text and crawl metadata in object storage (S3-compatible). This separates expensive raw payloads from transactional records and enables reprocessing when extraction prompts or schemas change.

Vector storage: use pgvector or a dedicated vector database for semantic retrieval over startup/product/news/paper descriptions. Store embedding model/version with each vector so embeddings can be regenerated deterministically.

Graph storage: use Neo4j or another graph database for relationships such as startup → product, paper → GitHub repository, company → job and entity → aliases. Keep graph data derived from canonical IDs rather than free-text names.

10. Canonical Record Lifecycle

1. Discover	2. Crawl	3. Normalize	4. Extract	5. Resolve	6. Validate	7. Persist
URLs/tasks	HTML/text	dates + text	LLM JSON	canonical entity	schema + freshness	DB + raw + graph/vector

11. Observability

Emit structured logs for source, URL, record type, status code, latency, attempt, provider, chunk count and final outcome. Track metrics including crawl success rate, freshness acceptance rate, 429/413 counts, LLM provider success/fallback rate, extraction null-rate, dedup rate, queue depth and processing latency. Alerts should fire on sustained failure spikes or queue growth.

12. Security and Data Quality

API keys remain in environment variables and are never committed to Git. Source URLs are retained for traceability. Validate every canonical record against a schema before persistence. Do not fabricate missing fields. GitHub stars are fetched from GitHub and stored with the repository URL and collection timestamp rather than inferred from paper text.

13. Current Implementation Mapping

Component	Implementation
Async / resilient crawling	aiohttp + asyncio; Playwright Async fallback
Freshness	src/freshness/detector.py; strict 24-hour filtering
LLM extraction	src/llm/extractor.py; chunking + provider fallback
429 / 413	Backoff + jitter; provider fallback; smaller chunks
Entity resolution	src/entity/resolver.py; deterministic aliases
Research + GitHub	src/crawlers/papers.py + github.py
News / Jobs	src/crawlers/news.py + jobs.py
Startup acquisition	src/crawlers/startups.py; 1,730 records collected
Raw / processed data	data/raw/ and data/processed/

Design principle: every stage is independently retryable, observable and horizontally scalable. The same pipeline can move from thousands to 500k+ records by increasing worker and queue capacity rather than rewriting the core crawler or extraction logic.